

Simulador Web de Ascensor con Gestión de Estados, Eventos, Asincronía y Persistencia Local

Integrantes: Jose Contin, Miguel Matute y Luis Rodriguez

1. DESCRIPCION GENERAL DEL SIMULADOR

El proyecto consiste en una aplicación web interactiva del lado del cliente que simula de manera realista el funcionamiento de un ascensor en un edificio de 12 pisos. La aplicación ha sido desarrollada utilizando tecnologías web estándar (HTML5, CSS3 y JavaScript puro), sin requerir servidores, backend ni librerías externas.

El simulador implementa una arquitectura basada en el patron Modelo-Vista-Controlador (MVC) complementado con el patron Observador (Observer). La lógica gestiona el estado interno del ascensor, procesa eventos de usuario desde una botonera por piso, maneja una cola de solicitudes, ejecuta desplazamientos progresivos con temporizadores asincronos, y ofrece una estrategia de atencion optimizada de paradas en ruta (tipo SCAN). Ademas, cuenta con un panel interactivo de configuracion (velocidad y tiempo de parada) y persistencia total de datos mediante localStorage.

2. REGLAS DEFINIDAS

Para garantizar un funcionamiento coherente y seguro, se establecieron las siguientes reglas en el simulador:

- a. Limites fisicos: El ascensor opera dentro de un rango definido de pisos (mínimo 1, máximo 12) y solo puede encontrarse en un único piso a la vez.
- b. Movimiento progresivo: El ascensor no se teletransporta. El desplazamiento entre pisos ocurre de forma secuencial paso a paso (1 a 1), tomando un tiempo determinado por la velocidad configurada.
- c. Máquina de estados y direcciones:
 - Direcciones posibles: "subiendo", "bajando" o "detenido".
 - Estados posibles: "detenido", "subiendo", "bajando" o "atendiendo" (durante la pausa de apertura y atencion de pasajeros en un piso).

d. Validacion de llamadas:

- No se atienden llamadas dirigidas al piso actual si el ascensor ya está detenido en él.
- No se duplican solicitudes que ya estén registradas en la cola de llamadas.
- No se duplica la solicitud si coincide con el piso destino que ya se encuentra en curso.

e. Prioridad de paradas en ruta (Algoritmo SCAN): Si el ascensor está en movimiento hacia un destino y se solicita un piso intermedio que coincide con su trayectoria y dirección actual, el ascensor se detiene a atenderlo antes de continuar hacia su destino original.

f. Persistencia continua: Cada cambio de piso, llamada registrada o parada atendida se sincroniza de forma inmediata en el almacenamiento local para garantizar consistencia ante recargas de página.

3. CANTIDAD DE PISOS

El edificio cuenta con un total de 12 pisos (del piso 1 al piso 12)

4. ESTADO INICIAL DEL ASCENSOR

Al ejecutarse la aplicación por primera vez (sin datos previos en localStorage), el ascensor presenta el siguiente estado inicial por defecto:

- Piso actual: 1
- Piso destino: null (ninguno)
- Dirección: "detenido"
- Estado: "detenido"
- Movimiento en curso: false
- Cola de solicitudes: (vacía)
- Solicitudes atendidas: 0
- Historial de eventos: (vacío)
- Velocidad de desplazamiento: 1000 ms por piso (calibrable de 200 ms a 2000 ms)
- Tiempo de parada / espera: 2000 ms (calibrable de 0 ms a 5000 ms)

Si existen datos previos en localStorage, estos valores se inicializan automaticamente con el ultimo estado recuperado.

5. ESTRUCTURA DE DATOS UTILIZADA

El simulador emplea diversas estructuras de datos de JavaScript para organizar la información:

a. Arreglos (Arrays):

- Solicitudes: Arreglo numérico utilizado como cola con política FIFO (First-In, First-Out) para almacenar las llamadas pendientes. Se manipula con métodos como push() (encolar), shift() (desencolar), indexOf() (búsqueda de duplicados) y splice() (extracción de paradas intermedias).
- Historial: Arreglo de cadenas de texto que almacena el registro cronológico de todos los eventos del simulador.
- Observadores: Arreglo de funciones callback que implementa la lista de suscripción del patrón Observer para notificar cambios a la vista.

b. Objetos y JSON (clave-valor):

- Objeto de persistencia: Estructura que encapsula el estado completo: { pisoActual, solicitudes, solicitudesAtendidas, historial, velocidadMs, tiempoParadaMs }, convertida a cadena mediante JSON.stringify() y parseada mediante JSON.parse().
- Módulos globales (window.Ascensor y window.Vista): Objetos que exponen las interfaces públicas respetando la encapsulación y la separación MVC.

c. Variables primitivas:

- Numéricas: pisoActual, pisoDestino, solicitudesAtendidas, velocidadMs, etc.
- Cadenas: direccion, estado.
- Booleanas: enMovimiento.

6. EXPLICACION DEL ALGORITMO DE MOVIMIENTO

El movimiento del ascensor se basa en un flujo asíncrono no bloqueante implementado con temporizadores encadenados:

1. Activación: Al recibir solicitudes y encontrarse libre (enMovimiento == false), la función mover() extrae el primer piso de la cola (pisoDestino = solicitudes.shift()), define la dirección ("subiendo" o "bajando"), cambia el estado e invoca a iniciarMovimiento().
2. Avance piso a piso: iniciarMovimiento() programa la ejecución de la función avanzar() tras el retardo configurado en velocidadMs:
 - Modifica el piso actual sumando o restando 1 unidad según la dirección.
 - Guarda el nuevo piso inmediatamente en localStorage.
 - Notifica a la vista, la cual actualiza el display numérico y desplaza la cabina visual mediante transiciones CSS

3. Comprobacion de eventos en cada paso:

- Si `pisoActual == pisoDestino`: Se llega al objetivo final, invocando a `atenderParada(true)`.
- Si el piso actual está contenido en la cola de solicitudes (`solicitudes.indexOf(pisoActual) !== -1`): Se detecta una parada de camino en la misma dirección (optimización SCAN), invocando a `atenderParada(false)`.
- Si no se cumple ninguna condición anterior: Se programa el siguiente paso con `setTimeout(avanzar, velocidadMs)`.

4. Atención de parada:

- En `atenderParada()`, se remueve la solicitud de la cola, se incrementa el contador `solicitudesAtendidas`, se agrega la entrada al historial, se guarda en `localStorage` y se establece el estado "atendiendo".
- Tras expirar el tiempo `ParadaMs`:
 - * Si era el destino final: se limpia `pisoDestino`, se fija dirección en "detenido", `enMovimiento` en false y se llama nuevamente a `mover()` para procesar la siguiente llamada pendiente.
 - * Si era parada intermedia: se restablece la dirección de marcha y se continua el recorrido hacia el destino final con `iniciarMovimiento()`.

7. EXPLICACION DEL MANEJO DE SOLICITUDES

El flujo de atención de llamadas sigue los siguientes pasos:

1. Captura del evento: El usuario presiona el botón "Llamar" de un piso. El controlador intercepta el evento click, extrae el atributo `data-piso` y llama a `Ascensor.llamarAscensor(piso)`.
2. Filtro de validez: Se verifica que el piso no sea el actual (estando detenido), que no sea el destino en curso y que no se encuentre ya en la cola.
3. Encolamiento: Si la llamada es válida, se inserta al final del arreglo `solicitudes` con `push(piso)`.
4. Registro y difusión: Se genera el evento en el historial ("Llamada realizada al piso X desde piso actual Y"), se sincroniza con `localStorage` y se notifica a los observadores. La vista actualiza el panel y resalta visualmente el botón en estado activo.
5. Atención: Se invoca `mover()`, que inicia la marcha si el ascensor estaba inactivo o permite que la solicitud sea atendida en camino si queda en la ruta.

8. DATOS ALMACENADOS LOCALMENTE

La aplicación utiliza la clave "ascensor_datos" en el localStorage del navegador, almacenando un documento JSON con la siguiente estructura:

```
{  
  "pisoActual": 5,  
  "solicitudes": [8, 12],  
  "solicitudesAtendidas": 7,  
  "historial": [  
    "Llamada realizada al piso 5 desde piso actual 1",  
    "Llegada al piso 5",  
    "Llamada realizada al piso 8 desde piso actual 5"  
  ],  
  "velocidadMs": 800,  
  "tiempoParadaMs": 2000  
}
```

Al cargar la página (cargarDeStorage), se valida que el piso recuperado este entre 1 y 12, que las solicitudes pendientes sean coherentes y que los rangos de velocidad y tiempo de parada sean válidos, protegiendo al sistema de posibles alteraciones manuales.

9. JUSTIFICACION DE LA PERSISTENCIA UTILIZADA

Se selecciono localStorage debido a las siguientes razones técnicas:

- Naturaleza del proyecto: Al ser una aplicación puramente del lado del cliente(sin backend ni servidor de base de datos), localStorage proporciona un mecanismo estándar, persistente y directo.
- Formato estructurado JSON: La combinación de localStorage con JSON.stringify() y JSON.parse() permite guardar y reconstruir objetos y arreglos complejos de JavaScript en una sola operación.
- Continuidad de sesión: Permite que el simulador conserve su estado exacto, historial y cola de trabajo aun cuando el usuario cierre el navegador o recargue la página (F5).
- Rendimiento y simplicidad: Posee una API sencilla y eficiente para el volumen de datos del simulador

10. DIFICULTADES ENCONTRADAS

Durante el desarrollo se presentaron y resolvieron los siguientes retos técnicos:

- a. Inconsistencia de estado al recargar durante el movimiento: Si el usuario recargaba la página en medio de un trayecto, el temporizador en memoria se perdía, dejando mensajes de llamada huérfanos en el historial. Se solucionó guardando `pisoActual` paso a paso en cada avance y almacenando la cola de solicitudes pendientes para reanudar el movimiento automáticamente al cargar.
- b. Paradas intermedias sincronizadas con asincronía: Coordinar la detención en un piso intermedio sin perder el destino final original ni provocar colisiones entre temporizadores requirió desacoplar el bucle en funciones de avance (`iniciarMovimiento`) y atención (`atenderParada`).
- c. Separación estricta MVC: Evitar acoplar la lógica de JavaScript con el árbol DOM de HTML. Se logró aislando completamente las variables y algoritmos en `ascensor.js`, y delegando la manipulación de clases CSS, textos y tablas a `vista.js` mediante el patrón Observer.

11. MEJORAS POSIBLES

Como futuras ampliaciones y optimizaciones del simulador se proponen:

- a. Llamadas direccionales (Botonera doble): Agregar botones de "Subir" y "Bajar" en cada piso para que el ascensor solo recoja pasajeros que viajen en su misma dirección.
- b. Múltiples cabinas de ascensor: Implementar un sistema de despacho inteligente que gestione 2 o más ascensores en simultáneo, asignando la llamada a la cabina más cercana o con menor tiempo de espera estimado.
- c. Puertas animadas y señales acústicas: Integrar animaciones SVG/CSS para la apertura y cierre de puertas de cabina y sonidos mediante la Web Audio API.
- d. Migración a IndexedDB para analítica: Utilizar IndexedDB para guardar un histórico masivo con marcas de tiempo (timestamps) y generar reportes o gráficos de uso por hora y por piso.